基于下一代NPU的Ascend C算子编程概述

作者 黄金华、吴洋

时间 2026/03

https://gitcode.com/cann/asc-devkit

CANN

目录

Part 1 Ascend C 是什么

Part 2 下一代硬件架构概述

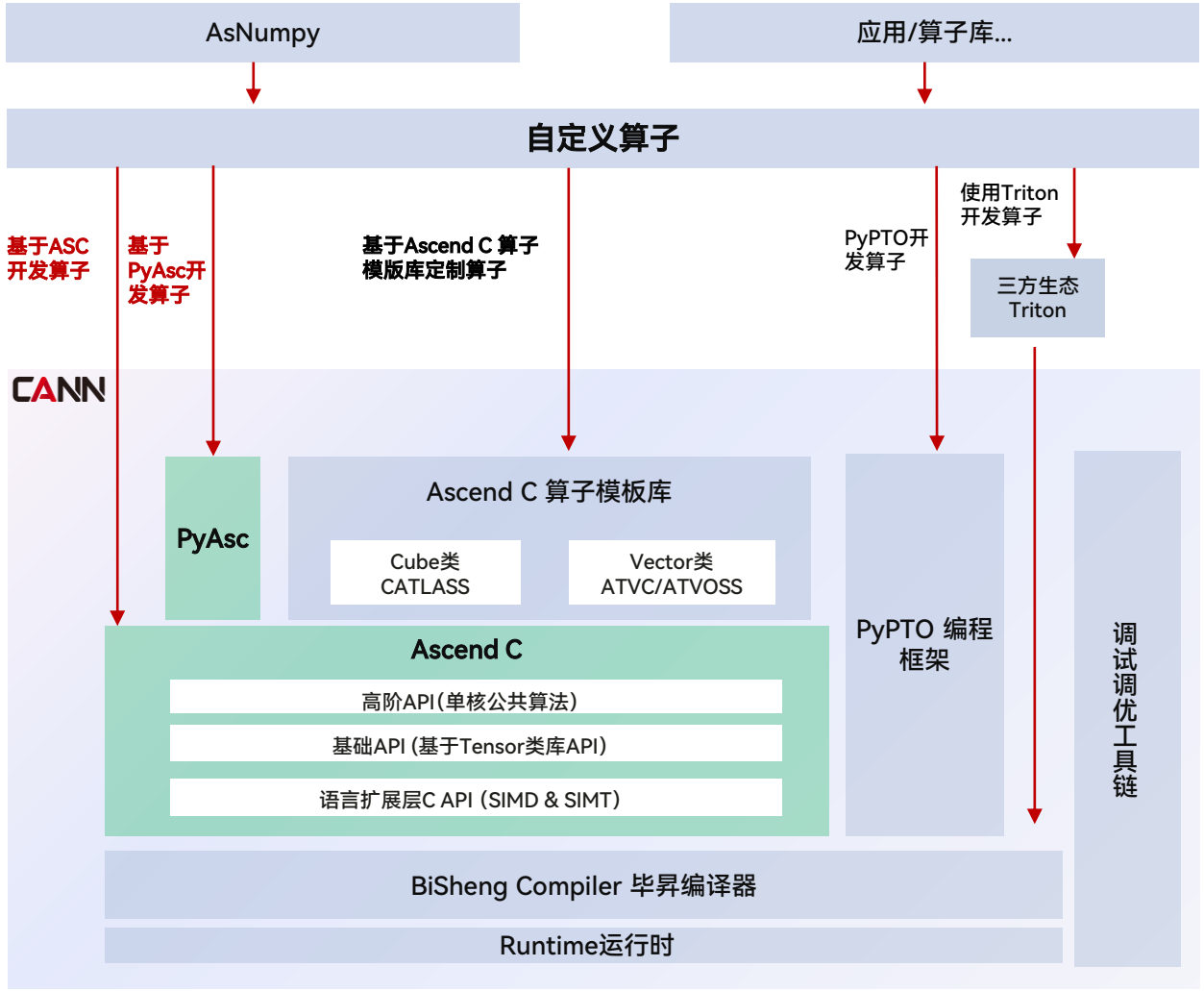
Part 3 下一代SIMT编程介绍

Part 4 下一代SIMD、SIMD/SIMT混合编程介绍

Part 5 下一代算子编译/调试调优简介

Part 6 当代算子到下一代迁移指导

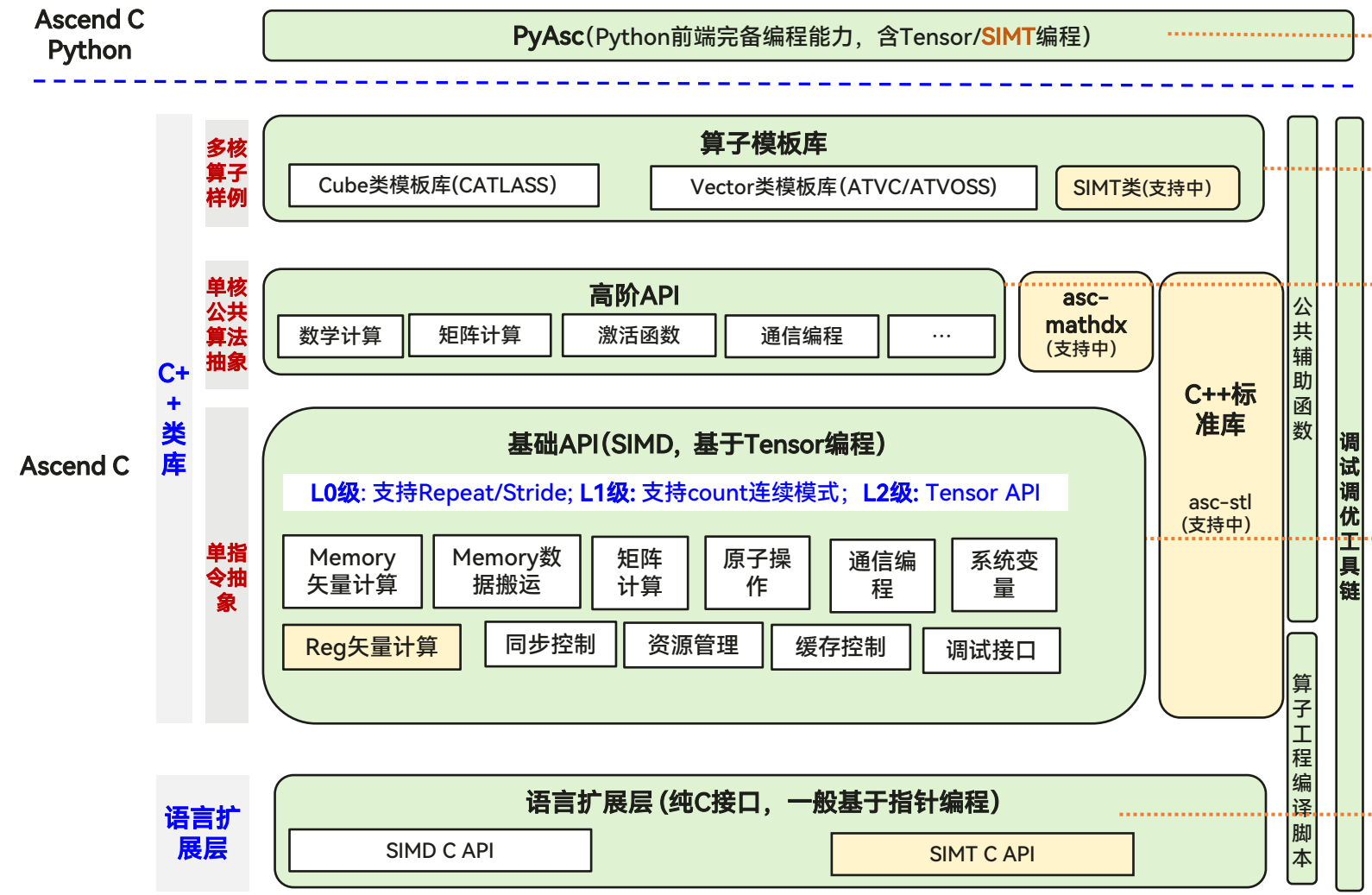
Ascend C: 基于C/C++扩展的昇腾算子编程语言



- 基于C/C++扩展，遵循C/C++标准规范；
- 提供底层芯片完备编程能力，支撑实现极致性能；
- 构建多级接口，满足多场景算子开发诉求，匹配业界开发习惯；
- 支持异构编程和<<<>>>直调
- 一套代码支持CPU/NPU孪生调试

https://gitcode.com/cann/asc-devkit
https://gitcode.com/cann/asc-tools

Ascend C算子编程：构建多层级API，支撑多维场景算子开发诉求



Python：提供**Python完备编程能力**，完善Python编程生态

C++类库：基于**模板提供算子完整实现**，简化Tiling开发，**支撑用户自定义修改**

C++类库：提供**单核公共算法**的实现，简化单核算法实现

C++类库：**基于Tensor接口**，支持Cube和Vector编程，支撑芯片**完备编程能力**；Tensor新增支持Layout，提升Tensor编程能力；下一代新增支持**寄存器Tensor**完备编程能力

C语言：**提供语言扩展层C API**，支持与业界一致的SIMD编程体验(下一代新增SIMT、SIMT&SIMD混合编程能力)

CANN

目录

Part 1 Ascend C 是什么

Part 2 下一代硬件架构概述

Part 3 下一代SIMT编程介绍

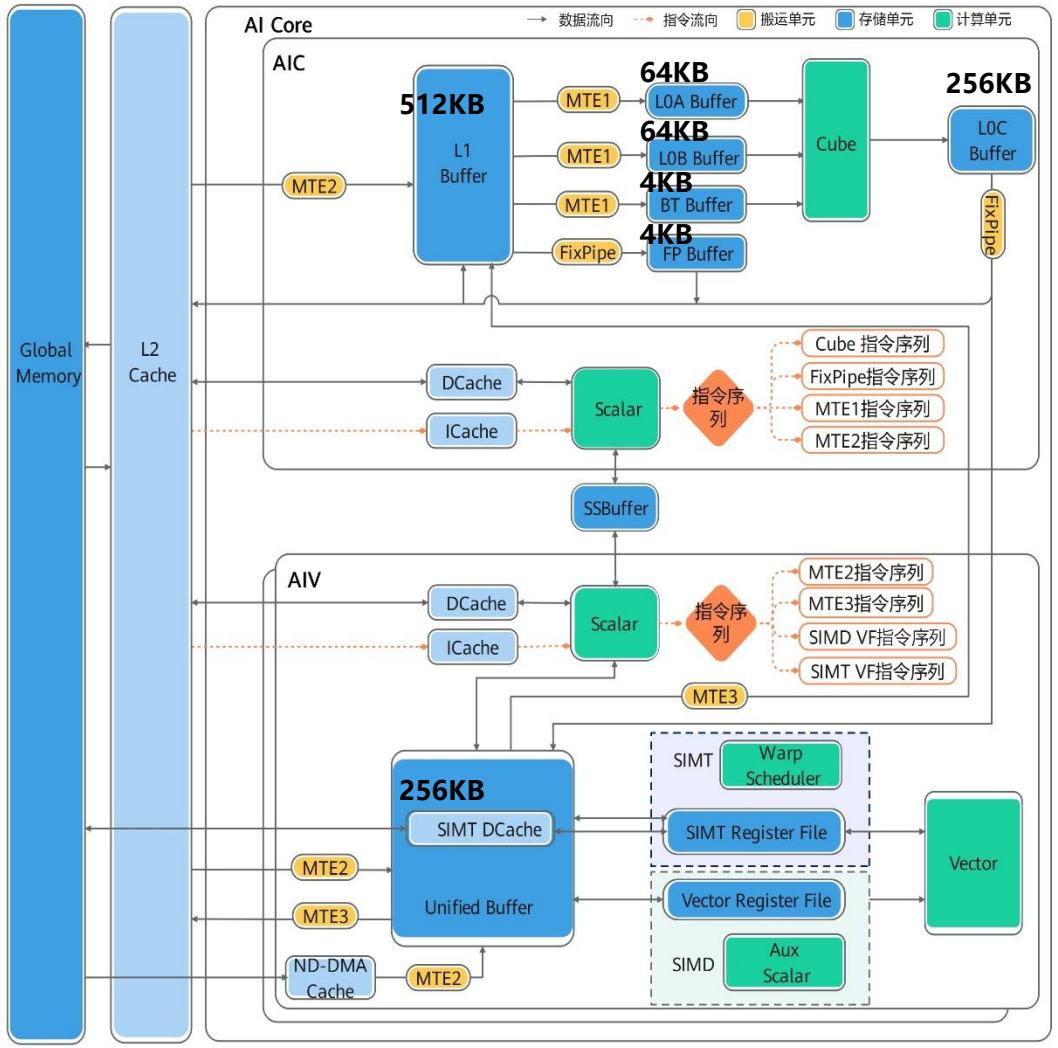
Part 4 下一代SIMD、SIMD/SIMT混合编程介绍

Part 5 下一代算子编译/调试调优简介

Part 6 当代算子到下一代迁移指导

下一代硬件架构概述

相比当代A2/A3，下一代AIV新增SIMT编程能力，可支持SIMT或SIMD/SIMT混合编程



AIV:

- 可支持两种工作模式：SIMT和SIMD，以VF(Vector Function） 为粒度进行切换；同一时刻不能同时支持SIMD和SIMT；
- SIMT
  - 支持直接读写GM或UB (UB当做Shared Memory)
  - 一个SIMT Vector Function 运行一个 thread block，同一时刻一个AIV只能执行一个SIMT Vector Function
  - 每个AIV有4个Warp Scheduler, scheduler id = warp id % 4
  - 每个线程可使用的寄存器受线程个数限制，具体如右边所示
  - 最大支持**128KB** Data Cache，Data Cache直接复用UB作为Cacheline
- SIMD
  - 从A2/A3Memorybased架构切换到Regbased架构，增加SIMD Register File存储层次，可以直接对芯片的Vector寄存器Register进行操作，实现更大的灵活性和更好的性能。数据从Unified Buffer搬运到Register进行计算，产生的中间结果可以不用传回Unified Buffer，直接在寄存器计算；

线程个数	寄存器个数
2048	16
1024	32
512	64

AIC:

- 新增MX FP4/MX FP8类型；
- 增强单核算力，增大核数；
- 不再支持s4类型；不支持4:2稀疏矩阵；

数据通路:

- 增加L0C Buffer -> Unified Buffer、Unified Buffer <-> L1 Buffer的数据通路；
- 删除Global Memory -> L0A Buffer、Global Memory -> L0B Buffer的数据通路。删除L1 Buffer-> Global Memory的数据通路；
- MTE2 数据搬运能力增强：新增NDDMA搬运、新增Compact模式、增强搬运维度；
- SSBuffer，用于AIC和AIV的核间通信；

CANN

下一代编程模型概述以及适用场景

<table>
<tr><th colspan="2">编程模型</th><th>适用场景</th></tr>
<tr><td colspan="2">SIMT编程【新增】</td><td>① 适合离散场景或复杂分支场景下矢量算子开发;
② 适合熟悉业界SIMT的算子开发人员快速入手NPU芯片算子开发;</td></tr>
<tr><td rowspan="2">SIMD 编程模型</td><td>SIMD编程</td><td>① 纯矩阵类算子开发;
② 纯矢量算子或融合算子开发：矢量计算部分主要是连续计算场景;</td></tr>
<tr><td>SIMD + SIMT混合编程【新增】</td><td>① 纯矢量算子或融合算子开发：矢量计算部分既有离散计算场景(通过SIMT函数实现)又有连续计算场景(通过SIMD函数实现），利用SIMD和SIMT各自优势;</td></tr>
</table>

CANN

本章小结

- **下一代NPU迎来颠覆性升级：**
  - AIV引入SIMT，支持SIMD和SIMT两种模式，并以VF（Vector Function）为粒度完成切换；
  - AIV的SIMD引入Reg架构，其数据流变成GM<->UB<->Reg；
  - AIC新增支持MX FP4/MX FP8类型，并进一步增强核数和算力；
  - 数据通路进一步加强；
  - …

- **编程模型进一步升级：**
  - 新增支持SIMT编程模型，支撑矢量算子开发；
  - 除继续支持现有SIMD编程外，新增支持SIMD/SIMT混合编程；

CANN

目录

Part 1 Ascend C 是什么

Part 2 下一代硬件架构概述

Part 3 下一代SIMT编程介绍

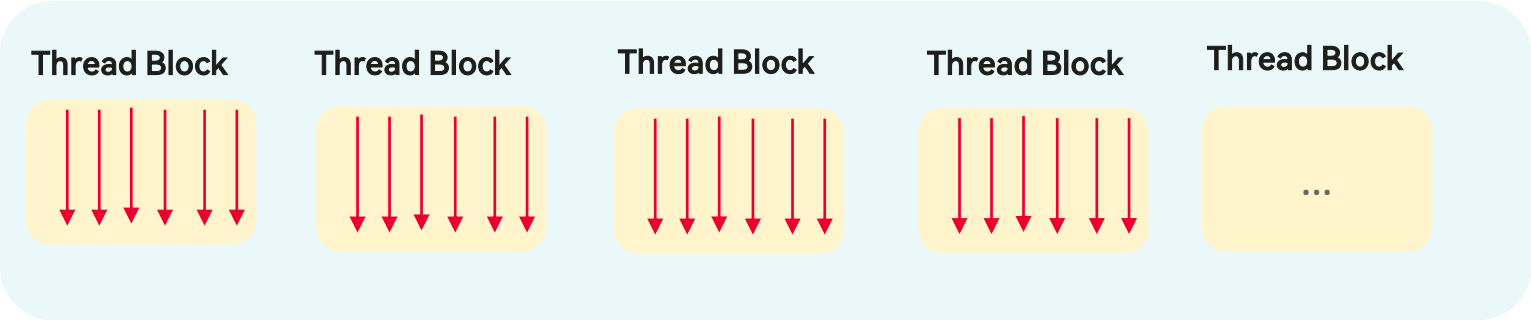
Part 4 下一代SIMD、SIMD/SIMT混合编程介绍

Part 5 下一代算子编译/调试调优简介

Part 6 当代算子到下一代迁移指导

下一代Ascend C SIMT编程(1)：遵循业界通用的线程架构和三级内存模型

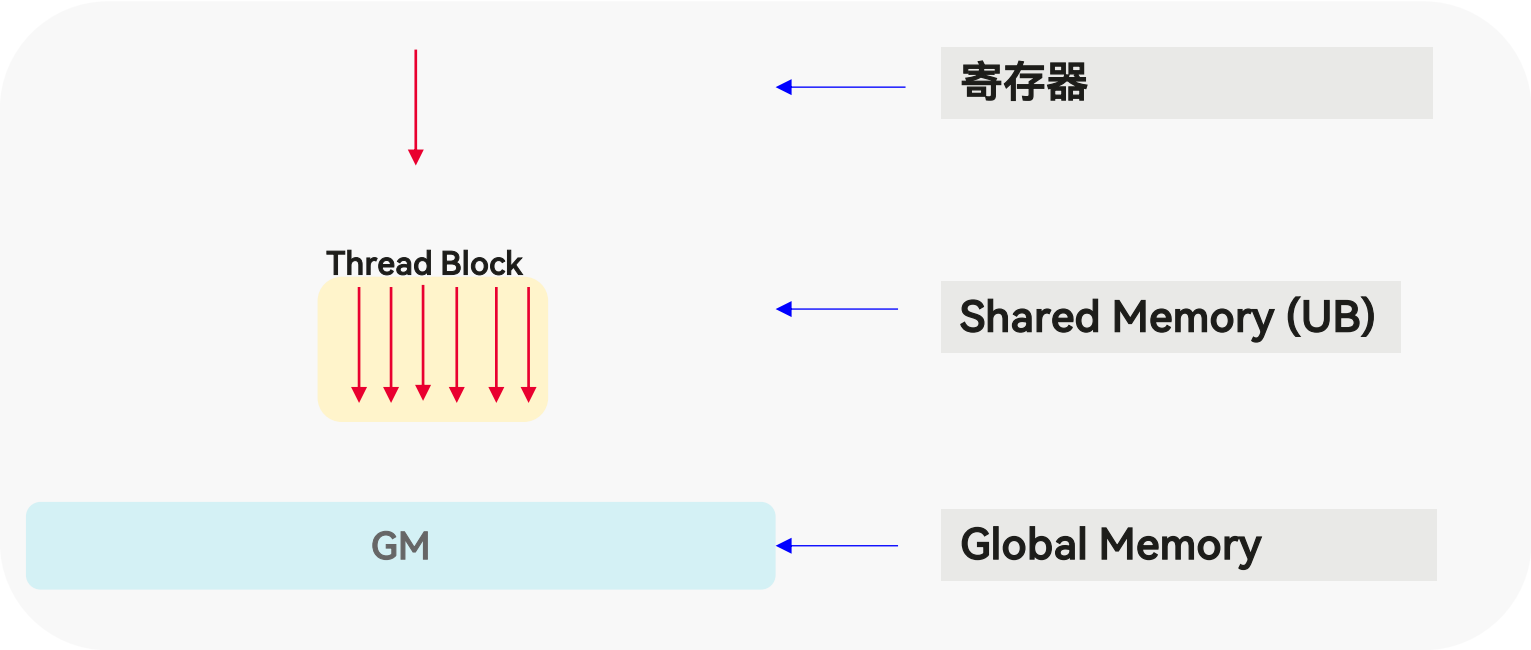
下一代 SIMT 线程架构



核心概念:

- block个数， gridDim (dim3)
- 一个block的线程个数, blockDim(dim3)
- 一次统一调度的线程组， warp

下一代 内存模型



- 一个SIMT Vector Function 运行一个 thread block，同一时刻一个AIV只能执行一个SIMT Vector Function
- 每个AIV有**4个**Warp Scheduler, scheduler id = warp id % 4
- 一个Thread Block 最多支持**2048**个线程;
- 一个Thread Block中SIMT RegFile总大小128K;

线程个数	寄存器个数
2048	16
1024	32
512	64

当前SIMT是业界SIMT功能子集，不支持Texture/Surface、cluster、TMA、 Independent Thread Scheduling 、Warp Specialization

下一代Ascend C SIMT编程(2)：UB内存分配与Kernel调用

支持业界常见的数组静动态内存分配

0

256K

静态内存

静态内存，通过数组分配 。
如__ubuf__ half staticBuf[1024];

动态内存

通过<<<>>>中**dynUBufSize** 指定的**动态内存**大小空间，可通过以下方式申请使用：
使用extern __ubuf__ half dynamicBuf[]; 动态数组分配;

预留空间

编译器和ASC**预留空间**，大小为8K;

Data Cache

SIMT 用于**Data Cache**的空间，必须大于或等于32K

```
dim3 numBlocks;  //GridDim;
dim3 threadsPerBlock; //BlockDim

kernel_name<<<numBlocks, threadsPerBlock,
dynUBufSize, stream>>>(args...)
```

DataCache = UB总大小(256K) – 静态内存 – 动态内存 – 预留空间(8K)

若DataCache小于32K，则RTS会校验报错

CANN

下一代Ascend C SIMT编程(3)： SIMT编程API

类别	ASC 编程API	业界SIMT编程接口	说明
内置变量	① gridDim、blockIdx、blockDim、threadIdx、warpSize	① gridDim、blockIdx、blockDim、threadIdx、warpSize	
内置函数/内存修饰符	① __global__、__aicore__ 、__host__ ② __ubuf__	① __global__ 、__device__、__host__ ② __shared__	
核函数定义	__global__ __launch_bounds(maxThreadsPerBlock) void myKernel(float* input, float*output, int size);	__global__ __launch_bounds(maxThreadsPerBlock, minBlocks) void myKernel(float* input, float*output, int size);	

核函数调用	dim3 numBlocks; //GridDim;
	dim3 threadsPerBlock; //BlockDim
	numBlocks 和 threadsPerBlock 同时支持int或dim3 类型;
	kernel_name<<<numBlocks, threadsPerBlock, dynUBufSize, stream>>>(args...)
	● dynUBufSize : 动态申请内存空间总大小
		dim3 numBlocks; //GridDim;
		dim3 threadsPerBlock; //BlockDim
		numBlocks 和 threadsPerBlock 同时支持int或dim3 类型;
		kernel_name<<<numBlocks, threadsPerBlock, shareMemSize, stream>>>(args...)	

Memory Fence	__aicore__ inline void asc_threadfence()、asc_threadfence_block()	__threadfence() 、__threadfence_block()、__threadfence_system	
同步函数	__aicore__ inline void asc_syncthreads()	__syncthreads()、__syncthreads_count、__syncthreads_and、__syncwarp	
Warp Vote	int32_t asc_all(int32_t predicate) 、asc_any(int32_t predicate)、 asc_activemask()	int __all(int predicate); int __all_sync(unsigned mask, int predicate);	

Warp Shuffle	__aicore__ inline int32_t asc_shfl(int32_t var, int32_t srcLane, int32_t width = warpSize)、asc_shfl_up、asc_shfl_down、asc_shfl_xor	int __shfl(T var, int srcLane, int width=warpSize);
		int __shfl_sync(unsigned mask, int var, int srcLane, int width=warpSize);
		__shf_up_sync、__shfl_down_sync、__shfl_xor_sync	

Warp Reduce	__aicore__ inline unsigned asc_reduce_add(unsigned val) 、asc_reduce_max、asc_reduce_min	unsigned __reduce_add_sync(unsigned mask, unsigned value) 、_reduce_min_sync、_reduce_max_sync	
原子函数	__aicore__ inline int_t asc_atomic_add(int *address, int val) 、asc_atomic_sub、asc_atomic_exch等_	int atomicAdd(int* address, int val)、atomicSub、atomicExch等	
Math函数	__aicore__ inline float asinf(float x)、__aicore__ inline float atanf(float x) 等	asinf(x) 、atanf(x) 等	因基础函数跟C标准库中math基本一致，因此跟随C Math函数命名保持一致
Launch Bound	___launch_bound(maxThreadsPerBlock)	__launch_bound(maxThreadsPerBlock, minBlocks)， minBlocks可选	

下一代Ascend C SIMT编程(4): 算子示例

ASC Reduce算子

```
__global__ void reduceKernel(float* input, float* output, int N) {
    extern __ubuf__ float sdata[]; // Shared memory for reduction
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Load data into shared memory
    if (i < N) {
        sdata[tid] = input[i];
    } else {
        sdata[tid] = 0.0f; // Handle out-of-bounds access
    }
    asc_syncthreads();

    // Perform reduction in shared memory
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        asc_syncthreads();
    }

    // Write result to global memory (only by thread 0 of each block)
    if (tid == 0) {
        output[blockIdx.x] = sdata[0];
    }
}

int main(int argc, char* argv[])
{
    ...
    // Launch the ASC Kernel (it also support dim3 for threadsPerBlock and blocksPerGrid)
    int threadsPerBlock = 256;
    int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;
    reduceKernel<<<blocksPerGrid, threadsPerBlock , dynUBufSize>>>(d_idata, d_odata, size);
    ...
    aclFinalize();
    return 0;
}
```

业界 Reduce算子

```
__global__ void reduceKernel(float* input, float* output, int N) {
    extern __shared__ float sdata[]; // Shared memory for reduction
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Load data into shared memory
    if (i < N) {
        sdata[tid] = input[i];
    } else {
        sdata[tid] = 0.0f; // Handle out-of-bounds access
    }
    __syncthreads();

    // Perform reduction in shared memory
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    // Write result to global memory (only by thread 0 of each block)
    if (tid == 0) {
        output[blockIdx.x] = sdata[0];
    }
}

int main(int argc, char* argv[])
{
    ...
   // Launch the CUDA Kernel (it also support dim3 for threadsPerBlock and blocksPerGrid)
   int threadsPerBlock = 256;
   int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;
   reduceKernel<<<blocksPerGrid, threadsPerBlock, smemSize>>>(d_idata, d_odata, size);
   err = cudaGetLastError();
    ...
   return 0;
}
```

本章小结

- **本章主要讲述如何进行SIMT编程，支撑矢量算子开发：**
  - 遵循业界通用的线程架构：Grid->Thread Block -> Thread；
  - 遵循业界通用的内存层级和内存分配方式；
  - 遵循业界通用的编程习惯和Kernel调用方式；
  - 主要支持矢量算子开发；SIMT编程只利用AIV，不涉及AIC；

CANN

目录

Part 1 Ascend C 是什么

Part 2 下一代硬件架构概述

Part 3 下一代SIMT编程介绍

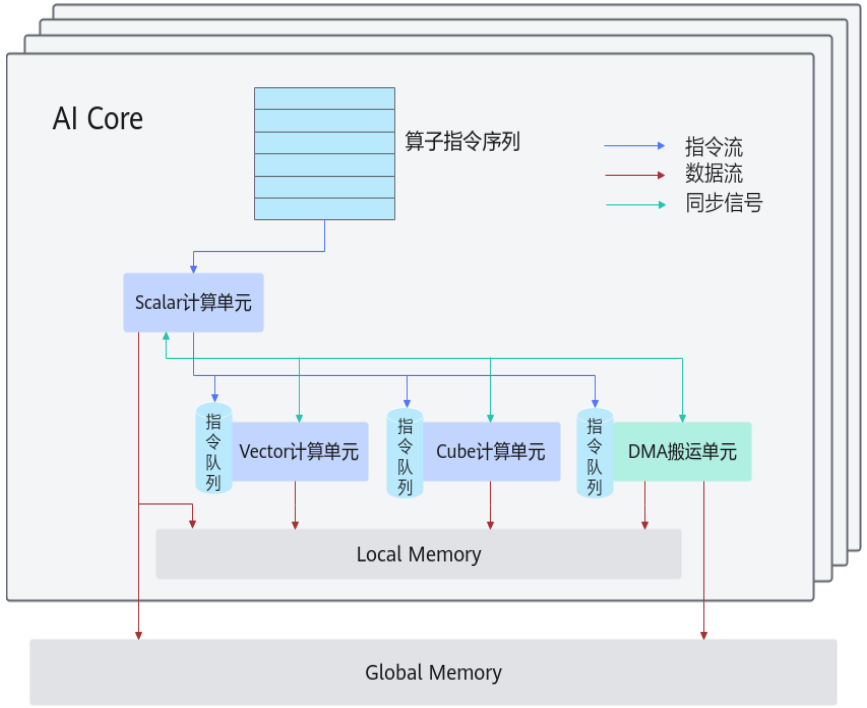
Part 4 下一代SIMD、SIMD/SIMT混合编程介绍

Part 5 下一代算子编译/调试调优简介

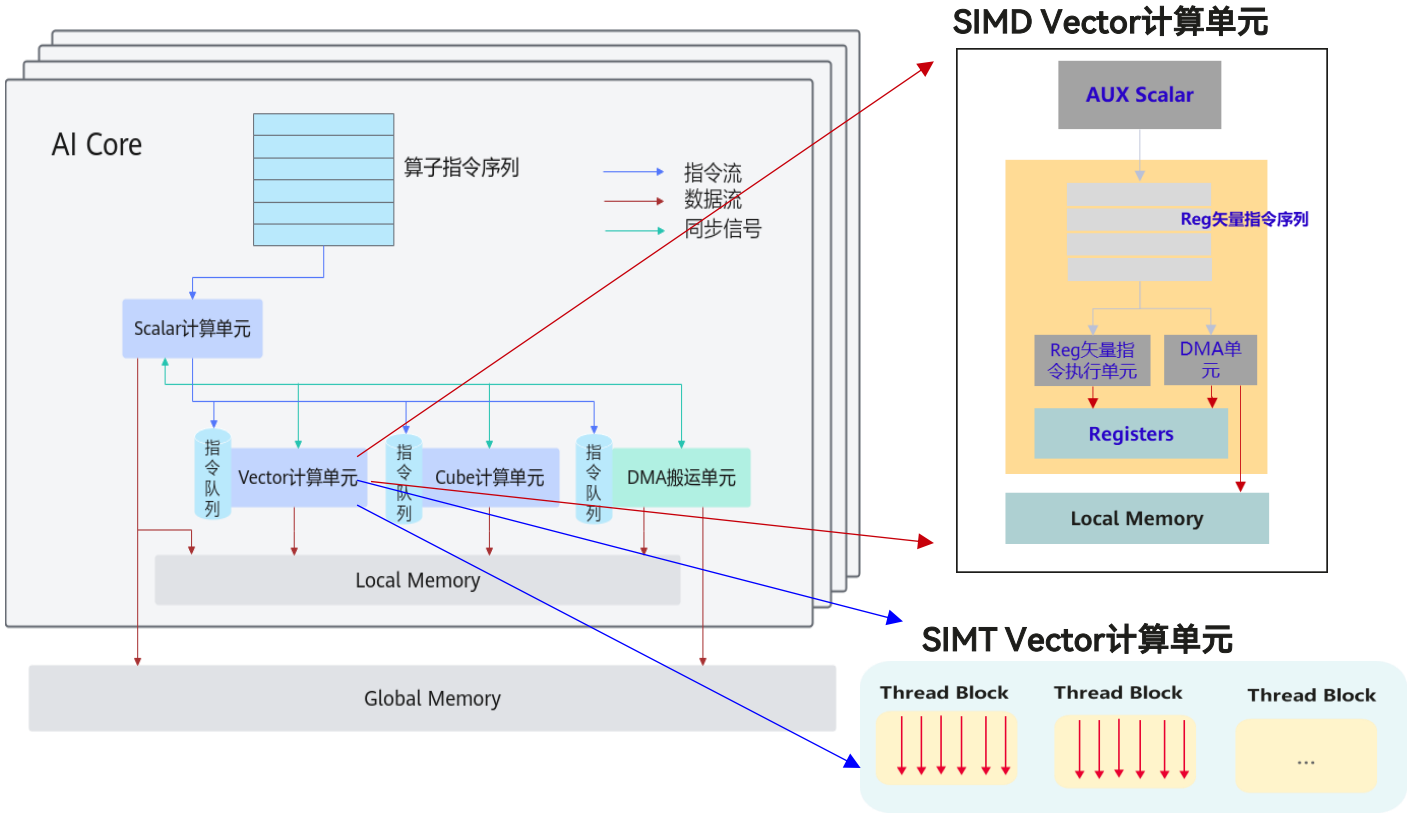
Part 6 当代算子到下一代迁移指导

Ascend C基于SIMD的抽象硬件架构演进

当代A2/A3 SIMD抽象硬件架构



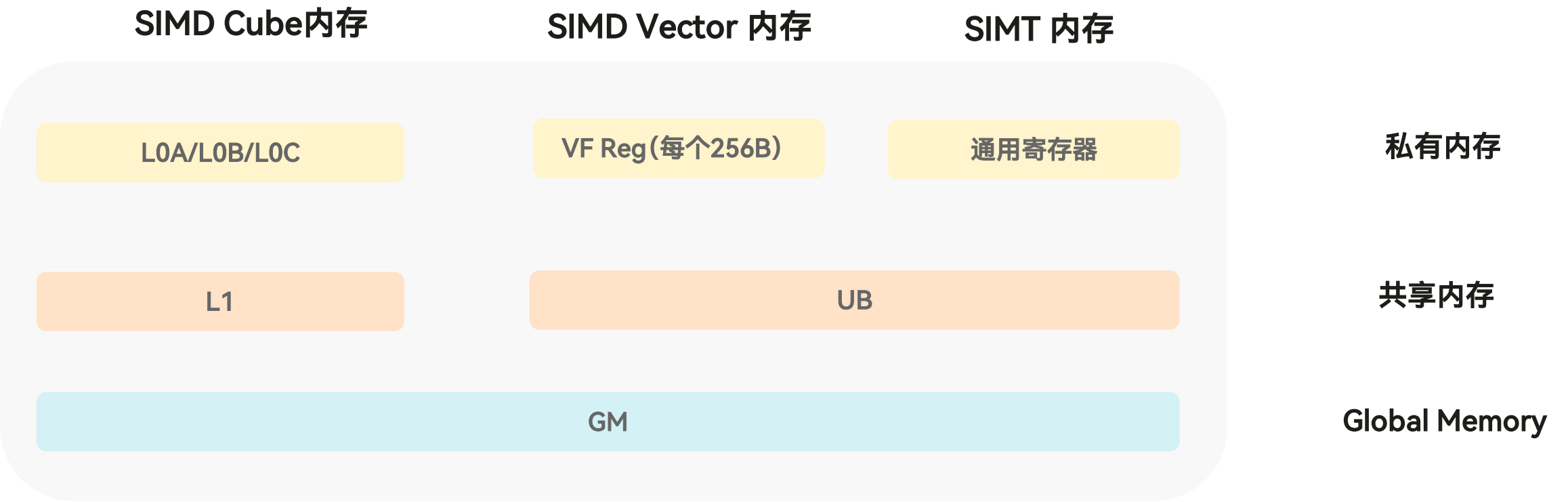
下一代 SIMD/SIMT 新同构抽象硬件架构



- **AICORE内核心组件**
  - 标量计算单元任务负责流程控制计算和地址计算（含Vector/Cube/DMA单元的指令发射）
  - Vector计算单元负责矢量计算；
  - Cube负责矩阵计算；
  - DMA负责数据搬运；

- **AICORE 核心变动点：**
  - Vector计算单元可支持SIMD或SIMT，以VF（Vector Function）粒度为单位进行切换；总体编程模型与上一代SIMD编程模型类似，只是一个Block可以调用SIMD VF（Vector Function）也可以调用SIMT VF；
  - SIMD Vector内部新增AUX Scalar，该Scalar单元仅负责SIMD VF函数内部标量计算，能力相比“Scalar计算单元”较弱，相关优化可参考VF融合优化指导；
  - SIMD Vector新增Reg内存层级。**为了保持兼容，**继续支持原来基于Local Memory(UB)层级的计算接口；**为了极致性能，**支持手工编写SIMD VF，先将数据从Local Memory(UB) 搬入到Reg，然后基于Reg完成矢量计算，最后将计算结果从Reg写回到Local Memory(UB)。
  - SIMT Vector以VF函数的方式支持SIMT，在**调用SIMT VF**时，需要**指定启动多少个线程；**

下一代Ascend C物理内存层级介绍

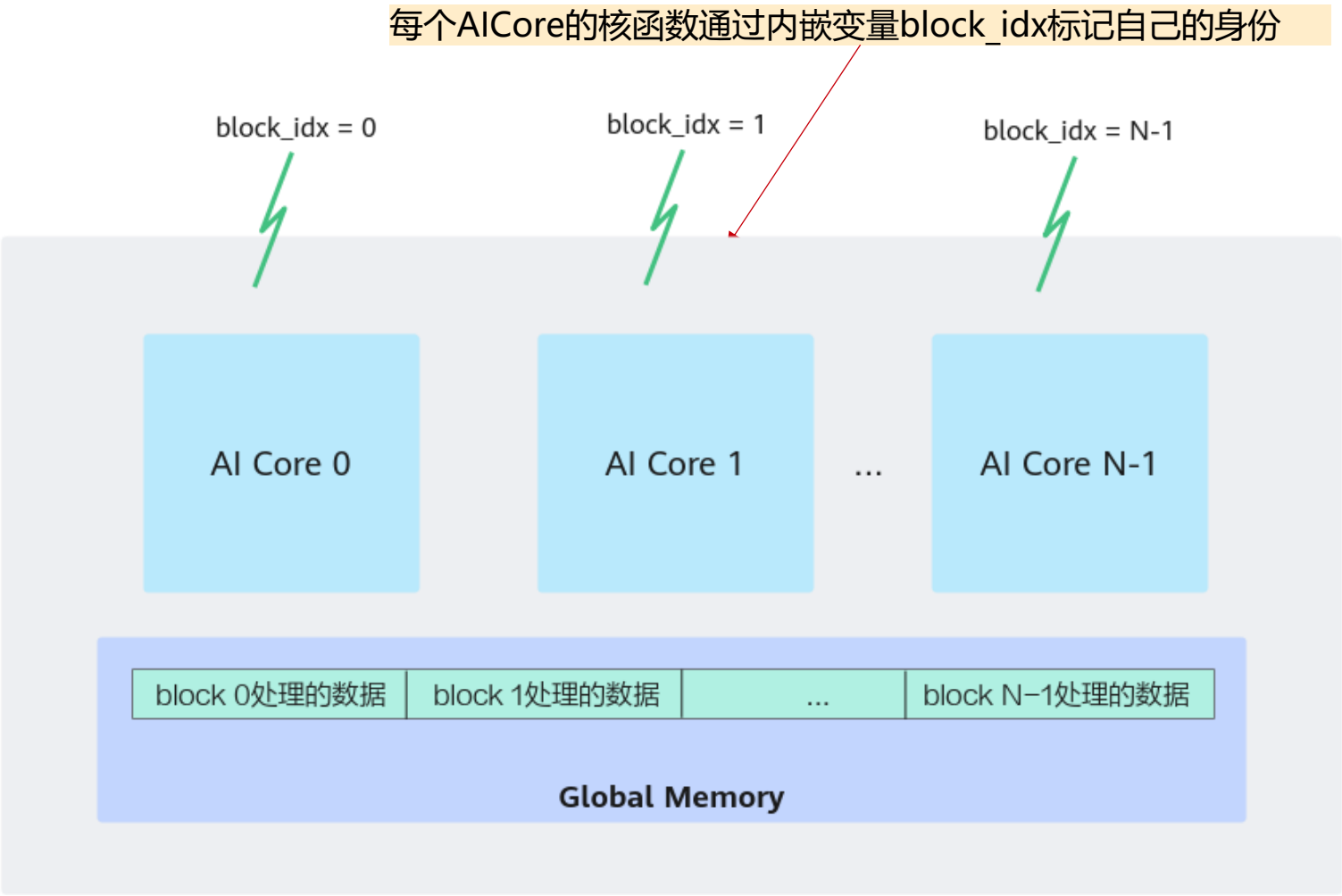


注：当代A2/A3支持GM/UB/L1/L0A/L0B/L0C，下一代新增VF Reg、SIMT通用寄存器(编译器自动分配)

CANN

Ascend C SIMD编程模型：SPMD(Single-Program Multiple-Data)

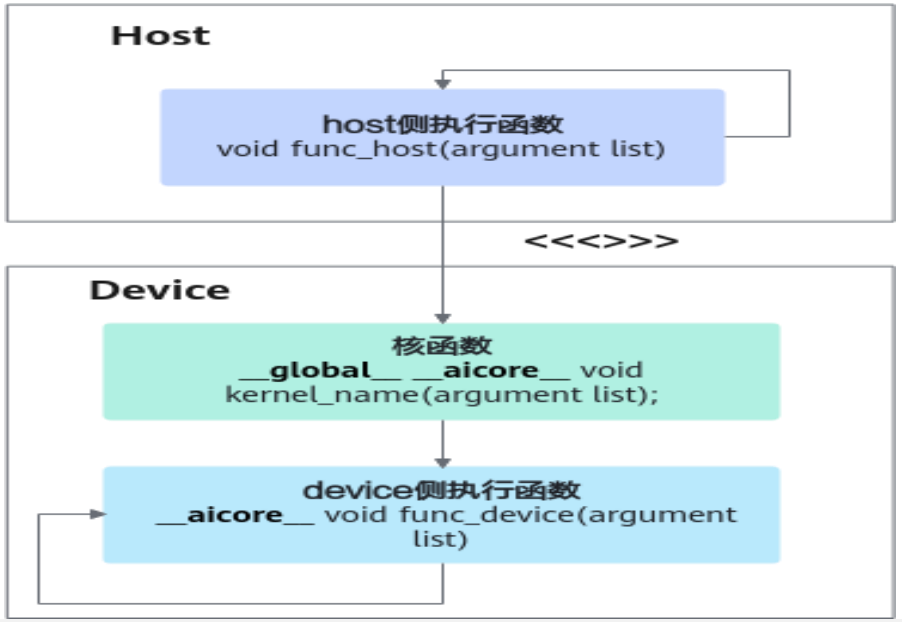
无论是SIMD还是SIMD/SIMT混合编程，其Kernel均遵循SPMD编程模型



CANN

下一代ASC SIMD核函数以及SIMD/SIMT VF函数调用方式

SIMD 核函数调用方式



Kernel调用方式：

kernel_name<<<numBlocks, dynUBufSize, stream>>>(argument list);

- **numBlocks**：规定了核函数将会在几个核上执行。每个执行该核函数的核会被分配一个逻辑ID，即block_idx，可以在核函数的实现中调用block_idx关键字
- **dynUBufSize(默认为0)**：指定的动态内存大小空间，可通过以下方式申请使用：
  ① 使用extern __ubuf__ half dynBuf[]动态数组分配；
  ② 通过Tque/Tpipe或LocalMemoryAllocator 分配；
- **stream**，类型为aclrtStream，stream用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序在device上执行

注：为防止内存冲突，方式1和方式2动态内存方式不能同时使用；

下一代 SIMD/SIMT混合编程VF调用方式

Kernel调用方式：

int numBlocks; // 只支持一维

mix_kernel<<<numBlocks, dynUBufSize, stream>>>(args...)

其中dynUBufSize 为动态UB内存大小，用户指定；

其中VF函数定义如下：

■ SIMD/SIMT VF函数定义：

```
__simt_vf__ __launch_bounds__(maxThreadsPerBlock) inline void
simtVFunction(…){}

__simd_vf__ inline void simdVFunction(…){}
```

■ 核函数定义以及SIMD/SIMT函数调用方式：

```
__global__ void mix_kernel() {
  …
  dim3 threadsPerBlock; // blockDim
  asc_vf_call<simtVFunction>(threadsPerBlock, …);
  …
  asc_vf_call<simdVFunction>(,…);
}
```

CANN

下一代ASC算子开发之SIMD编程范式概述

编程范式		适用场景
语言扩展层 C API(SIMD)		① 支持数组分配内存，支持指针计算接口，自主管理内存和同步，匹配业界开发习惯； ② 新增支持Reg矢量编程，通过定义vector_xxx 变量由编译器自动分配Reg；
基础API（基于Tensor）	Memory 矢量	① 兼容A2/A3，无需感知VF概念，但性能不是最优；
	Reg 矢量(新增)	① 新引入Reg层级，手工实现VF函数，可支撑实现极致性能；
	内存分配方式	① 支持通过Tque/Tpipe 自动管理同步和内存； ② 匹配业界习惯支持通过数组静态/动态分配UB内存，支持自主分配内存；待支持通过数组静态分配L1/L0A/L0B/L0C内存； ③ 支持LocalMemoryAllocator、Barrier自主管理内存和同步；
	Tensor 编程	① 下一代Tensor新增支持Layout，匹配业界Tensor编程体验；

CANN

下一代ASC算子开发之SIMD(1)：内存分配

UB 内存申请方式

0

256k

静态内存

静态内存，通过数组分配 。
如__ubuf__ half staticBuf[1024];

动态内存

通过<<<>>>中dynUBufSize 指定的**动态内存**大小空间，
可通过以下方式申请使用：
① 使用extern __ubuf__ half dynBuf[]**动态数组分配**；
② 通过**Tque/Tpipe或LocalMemoryAllocator** 分配；

注：为防止内存冲突，方式1和方式2动态内存方式不能同时使用；

预留空间

编译器和ASC**预留空间**，大小**为8K**；

其中dynUBufSize由用户通过<<<>>>指定：
kernel_name<<<numBlocks, dynUBufSize, stream>>>(args...)

L1/L0A/L0B/L0C 内存申请方式

静态内存

静态内存，通过数组分配（待支持） 。
如__ubuf__ half staticBuf[1024];

动态内存

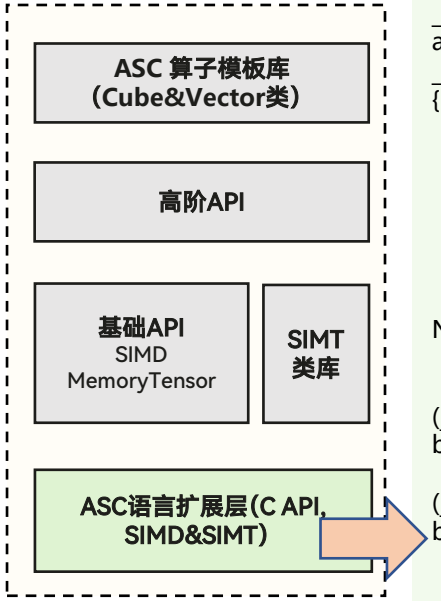
只支持**通过以下方式申请使用：**
① 通过**Tque/Tpipe或LocalMemoryAllocator** 分配；

注：因为<<<>>> 参数仅指定UB动态内存申请，因此L1/L0A/L0B/L0C 不支持通过数组分配动态内存

CANN

下一代ASC算子开发之SIMD(2)： 基于语言扩展层C API的Add示例

语言扩展层提供C风格API，支持数组分配内存，支持指针计算接口，自主管理内存和同步，匹配业界开发习惯



示例一：基于同步搬运/计算接口

```
__vector__ __global__ __aicore__ void
add_custom(__gm__ float* x, __gm__ float* y,
__gm__ float* z)
{
    asc_init();

    __ubuf__ float xLocal[TILE_LENGTH];
    __ubuf__ float yLocal[TILE_LENGTH];
    __ubuf__ float zLocal[TILE_LENGTH];

    uint32_t blockLength = TILE_LENGTH *
NUM_BLOCKS / asc_get_block_num();

    asc_copy_gm2ub_sync((__ubuf__ void*)xLocal,
(__gm__ void*)(x + asc_get_block_idx() *
blockLength), blockLength * sizeof(float));
    asc_copy_gm2ub_sync((__ubuf__ void*)yLocal,
(__gm__ void*)(y + asc_get_block_idx() *
blockLength), blockLength * sizeof(float));

    asc_add_sync(zLocal, xLocal, yLocal,
blockLength);

    asc_copy_ub2gm_sync((__gm__ void*)(z +
asc_get_block_idx() * blockLength), (__ubuf__
void*)zLocal, blockLength * sizeof(float));
}
```

示例二：基于异步搬运/计算+统一同步接口

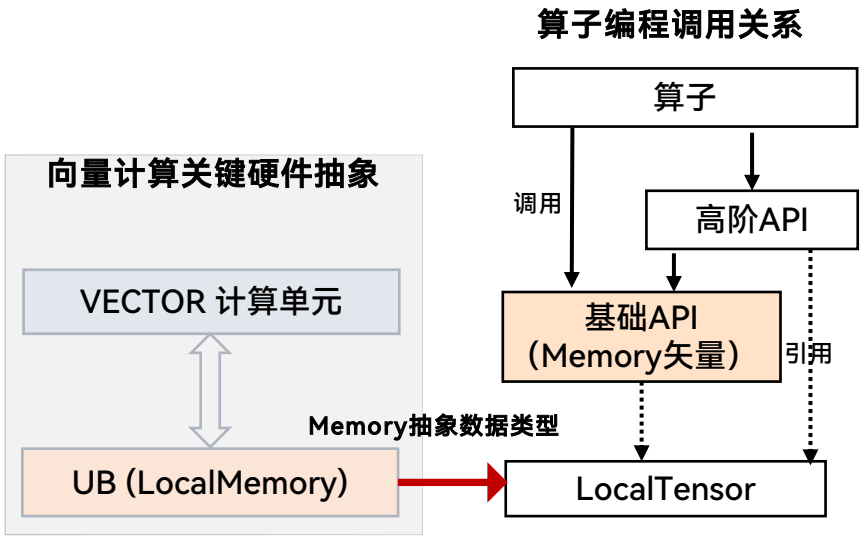
```
__vector__ __global__ __aicore__ void
add_custom(__gm__ float* x, __gm__ float* y,
           __gm__ float* z)
{
    asc_init();

    uint32_t blockLength = NUM_BLOCKS *
TILE_LENGTH / asc_get_block_num();

    __gm__ float* xGm = x + asc_get_block_idx() *
blockLength;
    __gm__ float* yGm = y + asc_get_block_idx() *
blockLength;
    __gm__ float* zGm = z + asc_get_block_idx() *
blockLength;

    __ubuf__ float xLocal[TILE_LENGTH];
    __ubuf__ float yLocal[TILE_LENGTH];
    __ubuf__ float zLocal[TILE_LENGTH];

    asc_copy_gm2ub((__ubuf__ void*)xLocal, (__gm__
void*)xGm, blockLength * sizeof(float));
    asc_copy_gm2ub((__ubuf__ void*)yLocal, (__gm__
void*)yGm, blockLength * sizeof(float));
    asc_sync();

    asc_add(zLocal, xLocal, yLocal, blockLength);
    asc_sync();

    asc_copy_ub2gm((__gm__ void*)zGm, (__ubuf__
void*)zLocal, blockLength * sizeof(float));
    asc_sync();
}
```

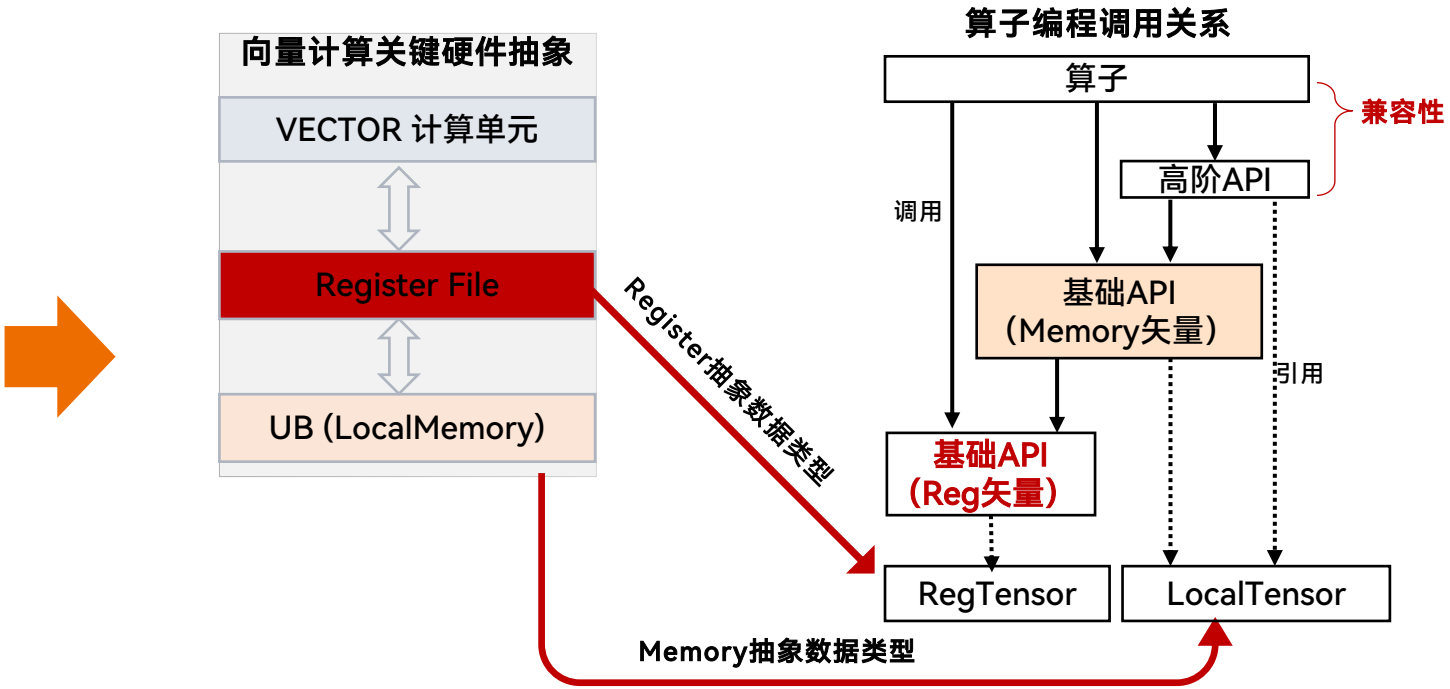
示例三：基于异步搬运/计算+ 精细化同步接口

```
__vector__ __global__ __aicore__ void add_custom(__gm__ float* x,
__gm__ float* y, __gm__ float* z)
{
    asc_init();
    __ubuf__ float xLocal[C_API_TILE_LENGTH];
    ...
    uint16_t burst_len = tileLength;
    for (uint32_t i = 0; i < C_API_TILE_NUM; i++) {
        if (i != 0) {
            asc_sync_wait(PIPE_V, PIPE_MTE2, EVENT_ID0);
        }
        burst_len = tileLength * sizeof(float) / C_API_ONE_BLOCK_SIZE;
        asc_copy_gm2ub(xLocal, xGm + i * tileLength, 0, 1, burst_len, 0,
0);
        asc_copy_gm2ub(yLocal, yGm + i * tileLength, 0, 1, burst_len, 0,
0);
        asc_sync_notify(PIPE_MTE2, PIPE_V, EVENT_ID0);
        asc_sync_wait(PIPE_MTE2, PIPE_V, EVENT_ID0);
        if (i != 0) {
            asc_sync_wait(PIPE_MTE3, PIPE_V, EVENT_ID0);
        }
        asc_add(zLocal, xLocal, yLocal, tileLength * sizeof(float) /
C_API_ONE_REPEAT_BYTE_SIZE, 1, 1, 1, 8, 8, 8);

        if (i != (C_API_TILE_NUM-1)) {
            asc_sync_notify(PIPE_V, PIPE_MTE2, EVENT_ID0);
        }
        asc_sync_notify(PIPE_V, PIPE_MTE3, EVENT_ID0);
        asc_sync_wait(PIPE_V, PIPE_MTE3, EVENT_ID0);
        asc_copy_ub2gm(zGm + i * tileLength, zLocal, 0, 1, burst_len, 0,
0);
        if (i != (C_API_TILE_NUM-1)) {
            asc_sync_notify(PIPE_MTE3, PIPE_V, EVENT_ID0);
        }
    }
}
```

https://gitcode.com/cann/asc-devkit /asc-devkit

下一代ASC算子开发之SIMD(3)：Reg矢量编程API





语言扩展层C API引入Reg矢量API

寄存器定义：

① **通用寄存器**：vector_half、vector_float等；

② **Mask寄存器**：vector_bool

数据搬运：

__aicore__ inline void asc_load(vector_half &dstReg, __ubuf__ half *srcUbAddr);

__aicore__ inline void asc_store(__ubuf__ half *dstUbAddr, vector_half &srcReg, vector_bool &mask);

计算：

Template <typename T = DefaultType, MaskMergeMode mode = MaskMergeMode::ZEROING, typename RegT>

__aicore__ inline void asc_add(vector_half &dstReg, vector_half &srcReg0, vector_half &srcReg1, vector_bool &mask);

其中： vector_bool为支持掩码部分数据计算

基础API引入Reg矢量API

抽象数据结构为**RegTensor**，基于RegTensor计算的API

数据搬运：

template <typename T = DefaultType, LoadDist dist = LoadDist::DIST_NORM, typename RegTU>__aicore__ inline void **Load**(RegTU &dstReg, __local_mem__ T *srcUbAddr); template <typename T = DefaultType, StoreDist dist = StoreDist::DIST_NORM, typename RegT>__aicore__ **inline void Store(__local_mem__ T *dstUbAddr**, RegT &srcReg, MaskReg &mask);

计算：

Template <typename T = DefaultType, MaskMergeMode mode = MaskMergeMode::ZEROING, typename RegT>

__aicore__ inline void Add(RegT &dstReg, RegT &srcReg0, RegT &srcReg1, MaskReg &mask);

其中：MaskReg：为支持掩码部分数据计算

下一代ASC算子开发之SIMD(4)： Reg矢量算子示例

示例一：基于语言扩展层SIMD C API(Reg矢量）示例

```
__simd_vf__ __aicore__ inline void SimdAdd(__ubuf__ half* dstPtr, __ubuf__
  half * src0Ptr, __ubuf__ half * src1Ptr, uint32_t calCount ) {
   constexpr uint16_t oneRepeatSize = asc_get_vf_len() / sizeof(half);
   uint16_t repeatTimes = ceil(calCount, oneRepeatSize);

   vector_half dstReg0, srcReg0, srcReg1;
   vector_bool maskReg;

   for (uint16_t i = 0; i < repeatTimes; i++) {
      // The length of mask will be equal or less than asc_get_vf_len()
      maskReg = asc_update_mask_b16(calCount);
      asc_load(srcReg0, src0Ptr + i * oneRepeatSize);
      asc_load(srcReg1, src1Ptr + i * oneRepeatSize);
      asc_add(dstReg0, srcReg0, srcReg1, maskReg);
      asc_store(dstPtr + i * oneRepeatSize, dstReg0, maskReg);
   }
}
```

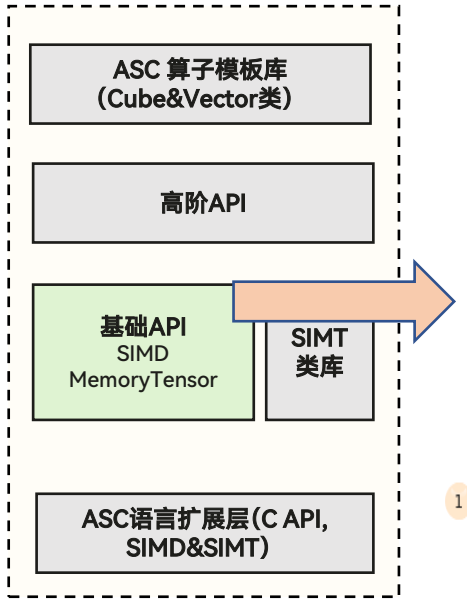
示例二：基于基础API Reg矢量计算

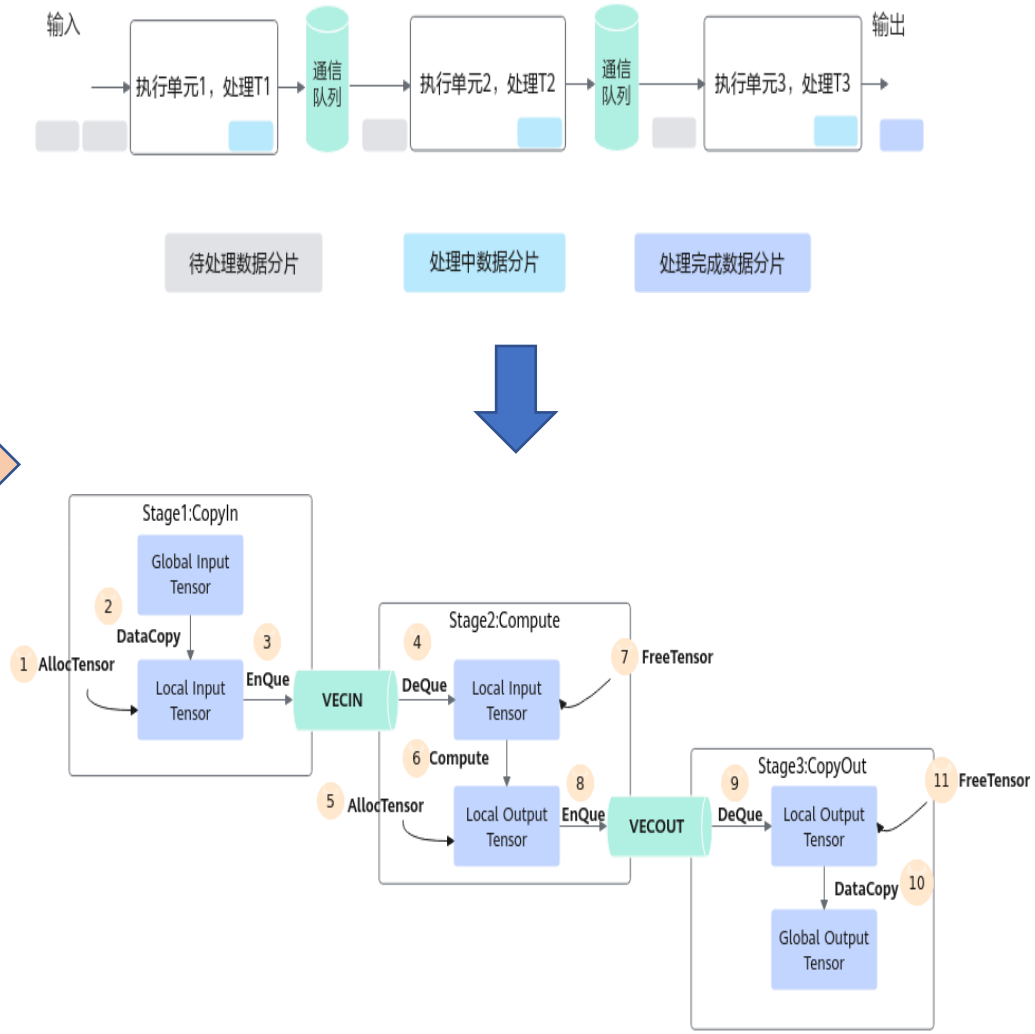
```
template<typename DTYPE>
__simd_vf__ __aicore__ inline void SimdAdd(__local_mem__ DTYPE* dstPtr, __local_mem__
   DTYPE* src0Ptr, __local_mem__ DTYPE* src1Ptr, uint32_t calCount ) {
   constexpr uint16_t oneRepeatSize = AscendC::GetVecLen() / sizeof(DTYPE);
   uint16_t repeatTimes = AscendC::CeilDivision(calCount, oneRepeatSize);

   AscendC::Reg::RegTensor<DTYPE > vDstReg0;
   AscendC::Reg::RegTensor<DTYPE> vSrcReg0;
   AscendC::Reg::RegTensor<DTYPE> vSrcReg1;
   AscendC::Reg::MaskReg maskReg;
   for (uint16_t i = 0; i < repeatTimes; i++) {
      // The length of mask will be equal or less than AscendC::GetVecLen()
      maskReg = AscendC::Reg::UpdateMask<DTYPE>(calCount);
      AscendC::Reg::Load(vSrcReg0, src0Ptr + i * oneRepeatSize);
      AscendC::Reg::Load(vSrcReg1, src1Ptr + i * oneRepeatSize);
      AscendC::Reg::Add(vDstReg0, vSrcReg0, vSrcReg1, maskReg);
      AscendC::Reg::Store(dstPtr + i * oneRepeatSize, vDstReg0, maskReg);
   }
}
```

CANN

下一代ASC算子开发之SIMD(5): 基于基础API的Tque/Tpipe范式算子示例

基础API 提供基于Tensor的C++类库API，支持基于Tque/Tpipe框架简化同步和内存管理，提升编程易用性

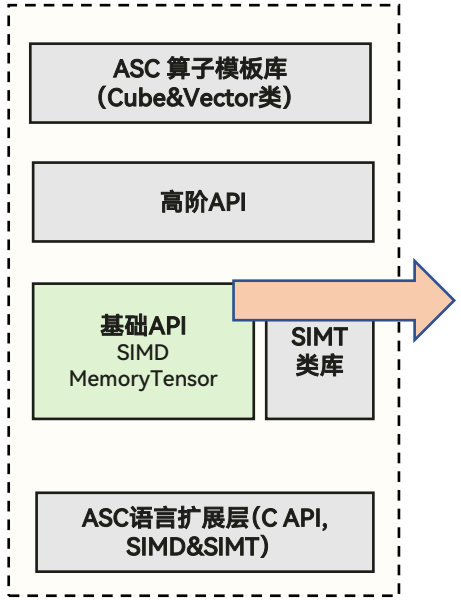





```
    __aicore__ inline void Process()
    {
        int32_t loopCount = this->tileNum * BUFFER_NUM;
        for (int32_t i = 0; i < loopCount; i++) {
            CopyIn(i);
            Compute(i);
            CopyOut(i);
        }
    }

private:
    __aicore__ inline void CopyIn(int32_t progress)
    {
        AscendC::LocalTensor<float> xLocal = inQueueX.AllocTensor<float>();
        AscendC::LocalTensor<float> yLocal = inQueueY.AllocTensor<float>();
        AscendC::DataCopy(xLocal, xGm[progress * this->tileLength], this->tileLength);
        AscendC::DataCopy(yLocal, yGm[progress * this->tileLength], this->tileLength);
        inQueueX.EnQue(xLocal);
        inQueueY.EnQue(yLocal);
    }
    __aicore__ inline void Compute(int32_t progress)
    {
        AscendC::LocalTensor<float> xLocal = inQueueX.DeQue<float>();
        AscendC::LocalTensor<float> yLocal = inQueueY.DeQue<float>();
        AscendC::LocalTensor<float> zLocal = outQueueZ.AllocTensor<float>();
        AscendC::Add(zLocal,xLocal, yLocal, this->tileLength);
        outQueueZ.EnQue<float>(zLocal);
        inQueueX.FreeTensor(xLocal);
        inQueueY.FreeTensor(yLocal);
    }
    __aicore__ inline void CopyOut(int32_t progress)
    {
        AscendC::LocalTensor<float> zLocal = outQueueZ.DeQue<float>();
        AscendC::DataCopy(zGm[progress * this->tileLength], zLocal, this->tileLength);
        outQueueZ.FreeTensor(zLocal);
    }
```

下一代ASC算子开发之SIMD(6): 基于基础API的自主管理内存/同步算子示例

基础API扩展能力，支持LocalMemoryAllocator、Barrier 自主管理内存和同步，提升自主管理能力



As-Is: TQue 范式编程

```
TPipe tpipe;
TQue<TPosition::VECIN, 1> vecInQue;
TQue<TPosition::VECOUT, 1> vecOutQue;

constexpr uint32_t tileLength = 1024;
// Initialize buffer size
tpipe.InitBuffer(vecInQue, 1,
tileLength*sizeof(half));
tpipe.InitBuffer(vecOutQue,1,
tileLength*sizeof(half));

for(int i=0; i < loop; i++) {
  //  Allocate Tensor
 auto in = vecInQue.AllocTensor<half>();
 DataCopy(in, gm+ i*1024, 1024);

  // Insert sync
 vecInQue.EnQue(in);
 vecInQue.DeQue(in);

 auto out = vecOutQue.AllocTensor<half>();

 Abs(out, in, 1024);

 vecInQue.FreeTensor(in);

  // Insert sync
 vecOutQue.EnQue(out);
 vecOutQue.DeQue(out);
 DataCopy(gm+i*1024, out, 1024);
 vecOutQue.FreeTensor(out);
}
```

To Be: 用户底层控制，同步与内存分离

```
LocalMemAllocator<MemType::UB> allocator;

constexpr uint32_t tileLength = 1024;
auto inLocalTensor = allocator.Alloc<half>(tileLength);
auto outLocalTensor =
allocator.Alloc<half>(tileLength);


Barrier<SyncScope::INTRA_CORE, mte3, mte2>
firstBarrier;
Barrier<SyncScope::INTRA_CORE, mte2, pipe_v>
secBarrier;
Barrier<SyncScope::INTRA_CORE, pipe_v, mte3>
thirdBarrier;

firstBarrier.Arrive(0);

for(int i=0; i<loop; i++){
   firstBarrier.Wait(0);
   DataCopy(in_ping, gm+ i *1024, 1024);
   secBarrier.ArriveAndWait(0);
   Abs(out_ping, in_ping, 1024);
   thirdBarrier.ArriveAndWait(0);
   DataCopy(gm+i*1024, out_ping, 1024);
   firstBarrier.Arrive(0);
}
firstBarrier.Wait(0);
```

CANN

下一代ASC算子开发之SIMD(7): 基础API(Tensor API) 扩展支持Layout

基础API 扩展支持Tensor API，Tensor支持Layout，简化基于Tensor的计算接口，匹配业界Tensor编程体验

ASC 算子模板库
(Cube&Vector类)

高阶API

基础API
SIMD
MemoryTensor

SIMT
类库

ASC语言扩展层(C API,
SIMD&SIMT)

```
template <typename... Shapes>
using Shape = Std::tuple<Shapes...>;
template <typename... Strides>
using Stride = Std::tuple<Strides...>;
```

Shape&Stride

```
template <typename ShapeType, typename StrideType>
struct Layout : private Std::tuple<ShapeType, StrideType> {
 // 构造函数
 __aicore__ inline Layout(Shape const& shape = {}, Stride const& stride = {})
 // 返回Shape
 template <int... I>
 __aicore__ inline Shape() {
 return get<0, I...>(static_cast<Std::tuple<Shape, Stride>&>(*this));
 }
};
```

Layout

```
template <typename Engine, typename Layout>
struct LocalTensor<TensorTrait<Engine, Layout>> {
   using Iter      = typename Engine::iterator;
   using engine_type  = Engine;
   using layout_type  = Layout;
 public:
   __aicore__ constexpr LocalTensor() {}
   __aicore__ constexpr LocalTensor(Engine engine, Layout la) : rep_(Std::move(la),
Std::move(engine)) {}
   __aicore__ uint64_t GetPhyAddr() const { return 1; };
 private:
   Std::tuple<layout_type, engine_type> rep_;
};
```

LocalTensor/GlobalTensor

搬运接口

向量计算接口

矩阵计算接口

```
// 1. 定义Problem 的Shape
auto probShape = MakeShape(m, n, k);

// 2. 定义Layout
auto gmStrideA = MakeStride(k, _1); // GMA Shape(m, k)
auto gmStrideB = MakeStride(_1, n); // GMB Shape(n, k)
auto gmStrideC = MakeStride(n, _1); // GMC Shape(m, n)
// 定义L1切分的Layout, 定长 1024 * 1024 * 256
auto l1M = _128, l1N = _128, l1K = _1024;
auto gm2L1Tiler = MakeShape(l1M, l1N, k);
auto layoutL1A = MakeLayout(MakeShape(l1M, l1K));
auto layoutL1B = MakeLayout(MakeShape(l1N, l1K));
auto l0M = _128, l0N = _128, l0K = _128;
auto layoutL0A = MakeLayout(MakeShape(l0M, l0K));
…

 // 基于数组申请静态内存
__cbuf__ half l1A[Cosize< layoutL1A >], l1B[Cosize< layoutL1B >];
__ca__ half l0A[Cosize< layoutL0A >];
…

// 3、构造片上Tensor
auto l1ATensor = MakeTensor(MakeL1memPtr(l1A), l1ALayout);
…

// 4、自定义数据搬运
auto gm2l1Copy = MakeCopy(CopyAtom<GM2L1Copy>{});
auto l12l0Copy = MakeCopy(CopyAtom<L12L0Copy>{});
constexpr static FixPipeTrait trait = {100};
auto l0c2OutCopy = MakeCopy(CopyAtom<L0C2OutCopy>{}, MakeTraits<trait>());

// 4、基于Layout的Tensor简化数据计算
…
Copy(gm2l1Copy, l1ATensor, globalBlkA(outMTile, outKTile));
Copy(gm2l1Copy, l1BTensor, globalBlkB(outNTile, outKTile));
Mmad(l0CTensor, l0ATensor, l0BTensor, outKTile == 0 && innerKTile == 0);
…
Copy(l0c2OutCopy, globalBlkC(outMTile, outNTile), l0CTensor);
…
```

下一代ASC算子开发之SIMD&SIMT混合编程(1)： 内存分配

UB 内存分配需要支持SIMT的Data Cache，L1/L0A/L0B/L0C 内存申请方式不变

0

256K

静态内存

静态内存，通过数组分配 。
如__ubuf__ half staticBuf[1024];

动态内存

通过<<<>>>中dynUBufSize **指定的动态内存大小空间，可通过以下方式申请使用：**
① 使用extern __ubuf__ half dynBuf[]**动态数组分配**;
② 通过**Tque/Tpipe或LocalMemoryAllocator** 分配;

注：为防止内存冲突，方式1和方式2动态内存方式不能同时使用;

预留空间

编译器和ASC**预留空间**，大小**为8K;**

Data Cache

SIMT 用于**Data Cache**的空间，必须大于或等于**32K;**
若没有SIMT，只有SIMD函数，则无需配置DataCache

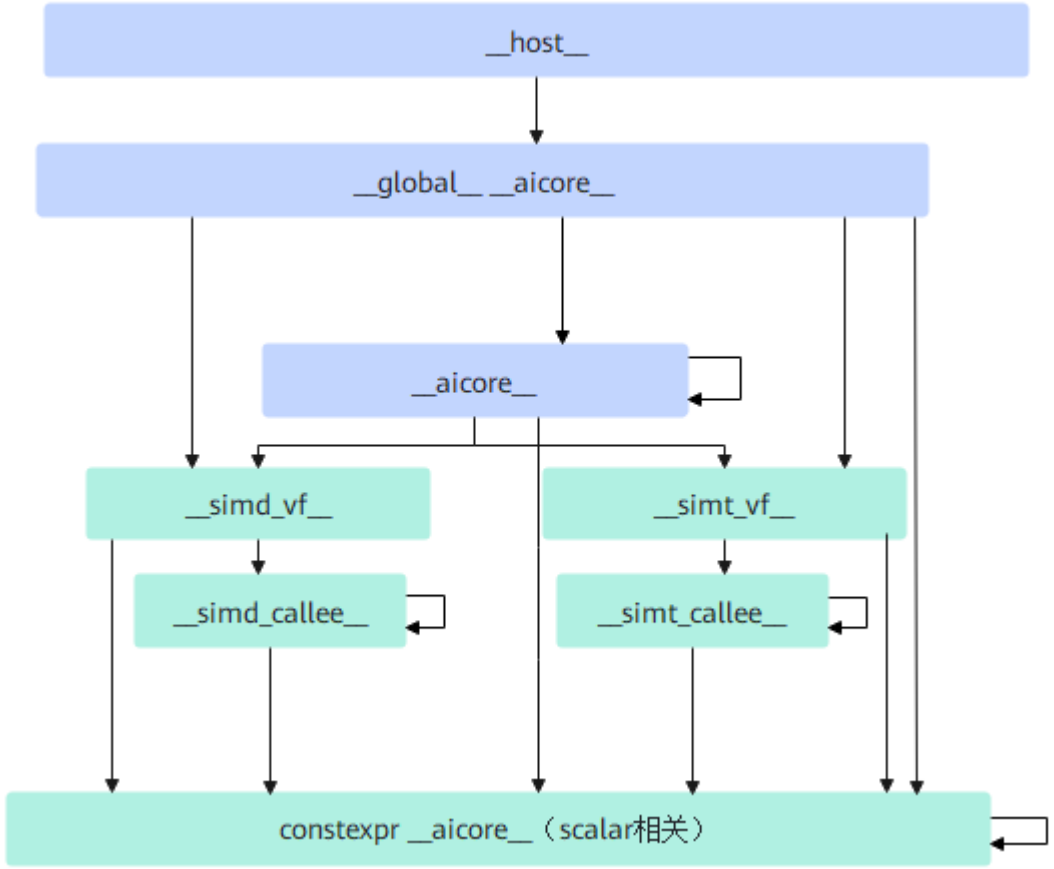
其中dynUBufSize由用户通过<<<>>>指定：

kernel_name<<<numBlocks, dynUBufSize, stream>>>(args...)

- **兼容历史SIMD:** 若dynUBufSize未配置(默认为0），此时不使用静态数组分配内存(兼容历史），此时动态内存大小为256K-8K;
- **新SIMD/SIMT场景，dynUBufSize必须配置**，则动态内存大小受指定大小限制；**此时可同时使用静态数组分配内存;**

若有SIMT函数，则DataCache必须大于或等于32k，则RTS会校验报错；若没有SIMT函数，则无需配置DataCache;

下一代ASC算子开发之SIMD&SIMT混合编程(2)： VF函数修饰符



- Vector计算单独同时支持SIMD和SIMT模式：
  - **__simd_vf__** 修饰的函数执行SIMD模式，其函数内部仅包含SIMD功能；
  - **__simt_vf__** 修饰的函数执行SIMT模式；

- 因为AIV引入了两个Scalar计算单元，其中一个传统的Main Scalar，另外一个是SIMD VF内部的AUX Scalar计算单元，两者能力不完全相同，此外SIMT也可执行Scalar计算，为便于编译器区分：
  - **__simd_callee__** 修饰的函数中Scalar计算在SIMD VF中AUX Scalar上执行；
  - **__simt_callee__** 修饰的函数中Scalar计算在SIMT VF中执行；
  - **__aicore__** 修饰的函数中Scalar计算在Main Scalar上执行；

CANN

下一代ASC SIMD/SIMT混合编程与SIMT编程差异总结

<table>
<tr><th>类别</th><th>SIMT 场景</th><th>SIMD/SIMT混合编程</th></tr>
<tr><td rowspan="3">内置变量</td><td rowspan="3">① SIMT: gridDim、blockIdx、blockDim、threadIdx、warpSize</td><td>SIMT VF (兼容SIMT程序员)： gridDim、blockIdx、blockDim、threadIdx、warpSize</td></tr>
<tr><td>SIMT VF(兼容SIMD老程序员)：block_num、GetBlockNum() 、block_idx、GetBlockIdx()、sub_block_idx、threadIdx、warpSize</td></tr>
<tr><td>SIMD VF: block_num、block_idx、sub_block_idx</td></tr>
<tr><td>内置函数/内存修饰符</td><td>① 基本跟业界保持一致</td><td>① 函数需要新增额外修饰符__simt_vf__、__simd_vf__、__simt_callee__ 、__simd_callee__
② 函数参数地址需要指定明确修饰：__ubuf__、__gm__</td></tr>
<tr><td>核函数定义</td><td>__global__
__launch_bounds(maxThreadsPerBlock) void myKernel(float* input, float*output, int size);</td><td>● 混合Kernel函数参考__global__ void myKernel(__gm__ float*,...)
● SIMD/SIMT VF 函数定义新增单独修饰符：__simt__vf__ / __simd_vf__等;</td></tr>
<tr><td>核函数调用</td><td>numBlocks 和 threadsPerBlock 同时支持int或dim3类型;
kernel_name<<<numBlocks, threadsPerBlock, dynUBufSize, stream>>>(args...)</td><td>numBlocks 支持int类型;
kernel_name<<< numBlocks, dynUBufSize, stream >>>(args...)</td></tr>
<tr><td>Memory Fence</td><td colspan="2" rowspan="7">SIMT和混合编程SIMT VF对SIMT API使用基本一致，主要是SIMD/SIMT混合编程时SIMT API指针参数地址需要指定明确的修饰符，如__gm__、__ubuf__</td></tr>
<tr><td>同步函数</td></tr>
<tr><td>Warp Vote</td></tr>
<tr><td>Warp Shuffle</td></tr>
<tr><td>Warp Reduce</td></tr>
<tr><td>原子函数</td></tr>
<tr><td>Math函数</td></tr>
<tr><td>Launch Bound</td><td colspan="2"></td></tr>
</table>

下一代ASC算子开发之SIMD&SIMT混合编程(3): 算子示例

SIMT VF函数示例

```
template <typename T>
__simt_vf__ __launch_bounds__(512) inline void SimtAdd(
__ubuf__ T* dst, __ubuf__ T* src0, __ubuf__ T* src1, uint32_t len)
{
   int  idx = blockDim.x * blockIdx.x + threadIdx.x;

   if (idx < len) {
      dst[idx] = src0[idx] + src1[idx];
   }
}
```

SIMD VF函数示例

```
__simd_vf__ inline void SimdAdd(__ubuf__ half* dstPtr, __ubuf__
   half * src0Ptr, __ubuf__ half * src1Ptr, uint32_t len ) {
    constexpr uint16_t oneRepeatSize = asc_get_vf_len() / sizeof(half);
    uint16_t repeatTimes = ceil(len, oneRepeatSize);

    vector_half dstReg0, srcReg0, srcReg1;
    vector_bool maskReg;

    for (uint16_t i = 0; i < repeatTimes; i++) {
       // The length of mask will be equal or less than asc_get_vf_len()
       maskReg = asc_update_mask_b16(len);
       asc_load(srcReg0, src0Ptr + i * oneRepeatSize);
       asc_load(srcReg1, src1Ptr + i * oneRepeatSize);
       asc_add(dstReg0, srcReg0, srcReg1, maskReg);
       asc_store(dstPtr + i * oneRepeatSize, dstReg0, maskReg);
    }
}
```

同时调用SIMT VF和SIMD VF的SIMD Kernel

```
template<typename T>
extern "C" __global__ __aicore__ inline void SIMDKernel(__gm__ T* x, __gm__ T *  y, __gm__ T * z, uint32_t len)
{
   // Allocate UB Memory
   __ubuf__ T dst0Local[512], src0Local [512], src1Local[512];

   ...
   // Use DataCopy Copy data from GM to UB
   asc_copy_gm2ub(src0Local, gmTensor, len);
  ...
   dim3 blockDim {32, 1, 1};
   // Execute SIMT Vector Function(VF) -> One VF, One Thread Block
   asc_vf_call<SimtAdd<T>>(blockDim, dst0Local, src0Local, src1Local, len);

   // Execute SIMD Vector Function(VF) -> Just for demo
   asc_vf_call<SimdAdd>(dst0Local, src0Local, src1Local, len);
}
```

SIMD & SIMT混合示例:

① 同一个算子同时可包含SIMD和SIMT，充分发挥各自的优势

② 通过VF 函数来区分SIMD和SIMT

CANN

本章小结

- **本章讲述如何基于SIMD编程模型进行编程**
  - 总体遵循SPMD（Single-Program Multiple-Data）编程，每个Block运行相同代码处理不同数据;

- **SIMD 编程要点：**
  - 内置变量： block_num、block_idx、sub_block_idx;
  - 核函数调用：kernel_name<<<numBlocks, dynUBufSize, stream>>>(args...);
  - 新增Reg矢量编程：
    ① Reg矢量计算需要在SIMD VF内部实现;
    ② 先在Kernel函数中将GM数据搬运到UB中，将UB地址作为参数传递给SIMD VF函数;
    ③ 然后在SIMD VF中，从UB读取数据到Reg，完成计算后将Reg数据写回到UB;
    ④ 其Kernel函数通过asc_vf_call 调用SIMD VF;
  - 基于纯C编程接口，支持与业界类似的纯C编程体验;
  - 基于C++类库接口，支持与业界类似的Tensor编程体验，支持自主管理同步/内存，或通过Tpipe/Tque自动管理同步/内存;

- **SIMD & SIMT 混合编程要点：**
  - SIMD VF内置变量： block_num、block_idx、sub_block_idx ;
  - SIMT VF内置变量：gridDim、blockIdx、blockDim、threadIdx、warpSize;
  - 核函数调用遵循SIMD Kerne调用方法：kernel_name<<<numBlocks, dynUBufSize, stream>>>(args...);
  - 核函数内部通过调用asc_vf_call 调用SIMD VF或SIMT VF函数;
  - 一个SIMT VF函数内部与SIMT编程一样;
  - 一个SIMD VF函数内部编程与SIMD编程一样;

CANN

目录

Part 1 Ascend C 是什么

Part 2 下一代硬件架构概述

Part 3 下一代SIMT编程介绍

Part 4 下一代SIMD、SIMD/SIMT混合编程介绍

Part 5 下一代算子编译/调试调优简介

Part 6 当代算子到下一代迁移指导

ASC支持异构编译+<<<>>>直调，提供与业界一致编程体验

- 对标业界，支持异构融合编译和<<<>>>直调；
- 通过后缀名“.asc”或 编译选项“-x asc”使能异构编译；

```
...
__global__ __vector__ void hello_world() {

    AscendC::printf("Hello World!!!\n");

}


int32_t main(int argc, char const *argv[])
{
  aclInit(nullptr);
  int32_t deviceId = 0;
  aclrtSetDevice(deviceId);
  aclrtStream stream = nullptr;
  aclrtCreateStream(&stream);
  constexpr uint32_t blockNum = 8;
  hello_world<<< blockNum, nullptr, stream>>>();
  aclrtSynchronizeStream(stream);
  aclrtDestroyStream(stream);
  aclrtResetDevice(deviceId);
  aclFinalize();
  return 0;
}
```

Device侧代码

Host/Device编译归一

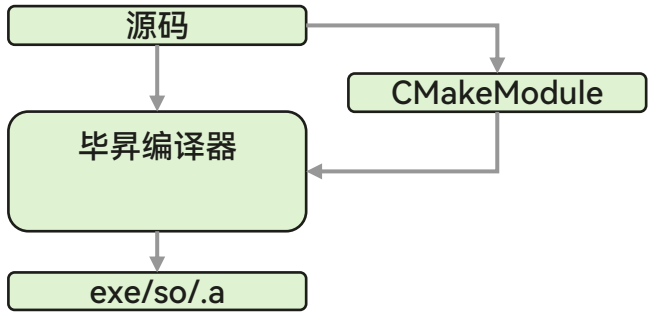
Host侧代码

使用命令行一键编译：

bisheng main.asc --npu-soc Ascend910B1

或基于标准cmake脚本

```
project(hello LANGUAGES ASC)
find_package(ASC)

add_executable(hello
    hello_world.asc
)
```



https://gitcode.com/cann/asc-devkit/asc-devkit/blob/master/examples/

Ascend C 调试调优概述

算子设计 → 算子编码 → 算子测试 → 算子调试 → 算子调优

算子设计

基于算子计算流以及芯片硬件参数进行理论建模：
- 极限性能分析
- Buffer复用关系
- Tiling设计
- 流水编排

算子编码

- 算子标准cmake项目工程的自动生成

算子测试

- 算子各层次接口ST标准cmake工程的自动生成
- 支持NPU和仿真环境测试
- 支持性能采集
- 支持精度校验

算子调试

- 支持print
- 支持孪生调试
- 支持上板debug
- 支持数据dump
- 内存检测
- 算子反汇编

算子调优

- Profiling性能数据采集，如MAC、带宽利用率等
- Roofline瓶颈分析
- 指令流水图

阶段	工具	使用场景
测试	msopst	辅助ST（System Test）测试
调试	CPU DEBUG（孪生调试）	**CPU 孪生（仿真）调试**，主要用于解决常见逻辑错误、异常访问等；
	msdebug	**上板调试**，可以设置断点、打印变量和内存、进行单步调试、中断运行（coredump后支持堆栈能力正在构建）
	mssantizer	**上板异常**内存检测
	msobjdump	**二进制解析**：解析算子ELF文件（Executable and Linkable Format），方便开发者直观获得kernel文件信息
	show_kernel_debug_data	**图模式下离线日志解析**：离线解析静态图的print日志信息
调优	msprof op	**性能采集和优化**：采集和分析运行在昇腾AI处理器上算子的关键性能指标

CANN

本章小结

- **算子编译与调用**
  - 算子文件后缀名支持 *.asc，支持同一个文件中编写Host+Kernel，支持混合编译;
  - 支持与业界一致的<<<>>> 调用;

- **算子调试调优:**
  - 算子编码：Vscode 插件;
  - 调试API: printf/DumpTensor/assert等;
  - 仿真调试：CPU 孪生调试;
  - 上板调试：msdebug;
  - 上板异常：mssantizer;
  - 性能调试：msprof;

CANN

目录

Part 1 Ascend C 是什么

Part 2 下一代硬件架构概述

Part 3 下一代SIMT编程介绍

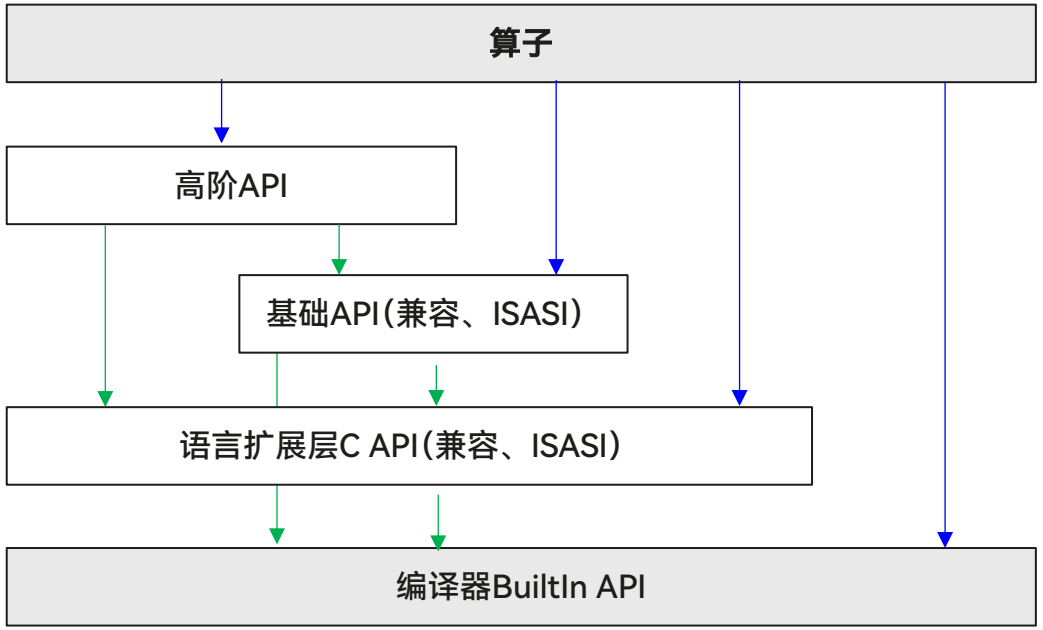
Part 4 下一代SIMD、SIMD/SIMT混合编程介绍

Part 5 下一代算子编译/调试调优简介

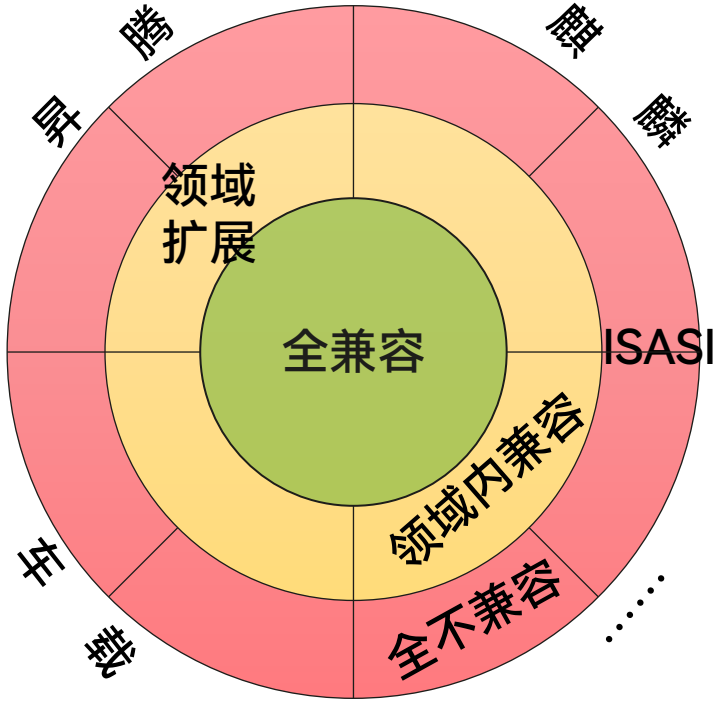
Part 6 当代算子到下一代迁移指导

Ascend C API 兼容性策略概述

Ascend C API层次结构



Ascend C API兼容策略



API层级	兼容策略目标
高阶API	高阶API保证同领域产品全平台兼容，特定领域扩展特性不兼容[1]
基础API	① 基础API区分兼容基础API和ISASI 基础API; ② 兼容API需要保证在所有产品上兼容; ③ ISASI API为体系架构相关API，对外不承诺跨芯片兼容， 如CUBE侧计算接口LoadData、MMAD等;
语言扩展层 C API	① C API可分兼容性/ISASI[2]
编译器BuiltInAPI	不保障兼容

注[1]：部分功能因兼容性代价较高，暂未完全支持兼容; [2]兼容性暂未支持，待规划

CANN

A2/A3到下一代不兼容基础API迁移指导

兼容说明

- **约95% A2/A3的ASC API（基础API和高阶API）可支持兼容，通过修改架构版本号重新编译实现兼容；**
- **约5% 基础API不支持兼容，可参考对应兼容迁移方案；**

硬件单元	A2/A3到下一代硬件能力变更	影响接口(基础API)	兼容迁移方案
搬运单元	删除L1Buffer到GM的数据通路	DataCopy/DumpTensor	1. cube only:在GM多分配一个单位矩阵，通过Mmad矩阵乘法计算输出到L0C Buffer，再从L0C Buffer通过Fixpipe搬运到GM 2. mix场景: 通过L1搬运到UB，再搬运到GM，需手动控制核间同步
	删除GM到L0ABuffer、L0BBuffer的数据通路	InitConstValue/LoadData	1. LoadData:实现GM-> L0A Buffer/L0B Buffer搬运需拆分成两步进行，先从GM搬运到L1 Buffer，再从L1 Buffer搬运到L0A Buffer/L0B Buffer 2. InitConstValue:先初始化L1 Buffer，再通过LoadData接口将将L1 Buffer上的数据搬运到L0A Buffer/L0B Buffer
计算单元	Cube计算单元不支持int4b_t类型	LoadData/LoadDataWithTranspose/Mmad	1. 算子侧通过MIX场景在vector core进行int4b_t到int8_t的转换，再通过UB搬运到L1后进行Mmad计算 2. 图层面可以在该算子前增加Cast节点进行int4b_t到int8_t的转换
	L0A分形变化ZZ->ZN	LoadData/LoadDataWithTranspose/Mmad	1. 非L0A Buffer切分的场景兼容220版本 2. L0A Buffer切分的场景需要根据新的分形重新计算stride
	Vector Core Membase架构切换到Regbase架构	基础API部分场景性能降低	1. 提供算子编译选项(-DNO_OVERLAP_IN_MULTI_REPEAT)优化性能(通过消除不必要的overlap插入的同步指令) 2. 极致性能可使用SIMD/SIMT VF编程
	硬件不支持Subnormal功能，当前使用软仿实现的Subnormal功能	Ln/Sqrt/Rsqrt/Div/Reciprocal/Exp	基础API层面提供精度兼容的config模板参数(通过软仿实现，性能降低)
	不支持4:2稀疏矩阵的计算	LoadDataWithSparse/MmadWithSparse	直接使用稠密矩阵计算接口
存储单元	删除L1Buffer空间的边界值设定	SetLoadDataBoundary	1. 接口参数boundaryValue设置为0时(默认场景）功能兼容 2. 如果需要在L1 Buffer上循环读取操作数，需要将对应的LoadData(Load3D)接口手动拆分成多条指令，手动续写
其他	删除UB异常调测接口	CheckLocalMemoryIA	该接口为调测接口，对功能无影响

注：1. Matmul高阶API：不支持int4b_t数据类型，不支持4:2稀疏特性；2. 当前不支持卷积计算类高阶API

A2/A3到下一代算子迁移案例

兼容案例一：修改Cmake中架构版本号直接编译实现兼容

```
target_compile_options(demo PRIVATE
    $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-2201>
    # NPU 架构号以正式发布的为准
    # $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-3510>
)
```

完整算子样例请参考：https://gitcode.com/cann/asc-devkit /asc-devkit/blob/master/examples/00_introduction/04_simple_operator/add_broadcast

兼容案例二：修改非兼容接口实现兼容

- 矩阵计算不再支持int4b_t
- 可通过Vector Core先将int4b_t转换成int8_t，然后再通过UB搬运到L1后进行Mmad计算

```
__aicore__ inline void Cast(…)
{
    // VectorCore Cast int4b_t 转换到 int8_t
    AscendC::Cast<half, AscendC::int4b_t>(tmpTensor, int4SrcLocalTensor,
AscendC::RoundMode::CAST_NONE, count * 2);
    AscendC::Cast<int8_t, half>(dstLocalTensor, tmpTensor,
AscendC::RoundMode::CAST_CEIL, count * 2);
    …
}
```

完整算子样例请参考：https://gitcode.com/cann/asc-devkit /asc-devkit/blob/master/examples/04_best_practices/10_compatibility_cases/matmul_s4_950

CANN

本章小结

- **本章讲述如何将A2/A3算子迁移到下一代**
  - 约95% A2/A3的ASC API（基础API和高阶API）可支持兼容，通过更改架构版本号重新编译可实现兼容;
  - 对部分不兼容的API，提供算子修改指导支持迁移;

CANN

附录

- Ascend C SIG: https://gitcode.com/cann/community/blob/master/CANN/sigs/Ascend C/README.md
- Ascend C 核心仓：https://gitcode.com/cann/asc-devkit
- CANN Ascend C 算子仓/算子模板库仓：https://gitcode.com/cann/
- Ascend C 编程指南：https://www.hiascend.com/cann/ascend-c

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann/asc-devkit